Square Integration Audit

Date: 2026-07-15
Method: Codebase, Prisma schema/migrations, services, API routes, UI, env, and test suites. Local sandbox DB diagnostics where available. No production DB assumed.
Related prior diagnostics: `docs/reports/square-multi-vendor-mapping-failure-diagnosis.md`

Executive summary

Verdict: Beta-ready with limitations — not production-ready for unsupervised Square vendors.

Open Order has a real, end-to-end Square path: OAuth → location → Square→OO catalog import/publish → location-scoped mappings → post-payment CreateOrder + EXTERNAL CreatePayment → `order.updated` webhook status sync. Multi-vendor orders resolve connections per `VendorOrder.vendorId`. **127 Square-related unit tests passed** in this audit run (23 files).

What is **not** launch-safe is the readiness gap that allows customers to pay and then fail at Square mapping, the lack of location-change mapping invalidation (Poke Sea class of failure remains possible), unpaid-orderable Square vendors when injection is not ready, incomplete money lifecycle (no Square refund/cancel money sync), and several schema/tenancy soft spots (no unique active connection; global Square-order-id lookup).

Top five risks

1. **Post-payment mapping failure (Critical)** — Public orderability does **not** require `squareOrderRoutingReady`. Readiness only requires **any** item mapping at the selected location (`count > 0`), not full coverage of sellable items. Customers can pay; Square submit fails per-line.
2. **Location / catalog drift (Critical)** — Switching/republishing locations leaves old `ProviderEntityMapping` rows active on other locations. Orders use selected location only. Poke Sea failure mode remains possible.
3. **No Square refund/cancel money sync (High)** — Stripe refunds do not reverse Square EXTERNAL payments; Square-side cancel is status-only.
4. **Connection uniqueness / active row selection (High)** — No DB unique on active Square connection; `getActiveSquareConnectionForVendor` uses `findFirst … orderBy updatedAt desc`. Concurrent reconnect races can leave multiple actives.

5. **Webhook → VO resolution is global by `squareOrderId` (Medium–High)** — `findVendorOrderIdBySquareOrderId` is not merchant/vendor-scoped (mitigated if Square order IDs are globally unique in practice).

Current architecture

Provider spine (data)

Model	Role	Path
`Vendor.orderRoutingMode`	Gate (`square` / `deliverect` / `manual_dashboard`)	`prisma/schema.prisma`
`Vendor.squareOrderRoutingEnabled`	**Deprecated** for readiness; mode is SoT	same
`VendorIntegrationConnection`	Active Square OAuth row: merchant, location, token refs, capabilities JSON	same
`IntegrationProviderCredential`	AES-GCM encrypted access/refresh tokens	same
`IntegrationOAuthStateNonce`	OAuth replay protection	same
`ProviderEntityMapping`	Item/modifier/category/order external IDs; **unique** `(vendorId, provider, type, internalId, externalLocationId)`	same
`ProviderWebhookEvent`	Inbound Square event log; unique `(provider, externalEventId)`	same
`VendorOrder.square*`	`squareOrderId` , attempts, errors, payload/response JSON	same
`MenuImportJob` / `MenuVersion`	`SQUARE_CATALOG_PULL` → canonical `square_catalog_v1` → publish to live tables	same

Migrations: `20260706120000_integration_provider_foundation` through `20260709120000_square_webhook_status_source` (see Prisma migrations folder).

Runtime flow (implemented)

```
Vendor Connect Square
  GET /api/vendor/[vendorId]/square/oauth/start
    → signSquareOAuthState (HMAC, AUTH_SECRET)
    → Square authorize
  GET /api/integrations/square/oauth/callback
    → verifySquareOAuthState + consumeSquareOAuthStateNonce
    → canManageVendor(state.vendorId)
    → completeSquareOAuthForVendor → storeIntegrationProviderTokens
    → upsertSquareConnection (merchant + locations)
Location select (if multi-location)
  selectSquareLocationForVendor
Catalog
  importSquareCatalog / syncSquareCatalogMappings (vendorId + locationId)
  publish → applyCanonicalMenuToLiveTables (soft-disable missing items)
Customer pays (Stripe)
  post-payment.service → for each VendorOrder:
    routing.service submitVendorOrder
      → submitVendorOrderToSquare
        → assertSquareOrderRoutingReady(vendorId)
        → getActiveSquareConnectionForVendor(vendorId)
        → mapVendorOrderToSquareCreateOrder (PEM by vendor + location)
        → createSquareOrder (idempotency oo:sq:order:{voId})
        → createSquareExternalPayment (oo:sq:pay:{voId}, source EXTERNAL)
Status back
  POST /api/webhooks/square (order.updated, HMAC verify)
    → applySquareOrderStatusSync → mapSquareOrderSnapshotToVendorStatus
    → VendorOrder fulfillment/routing update (monotonic)
```

Key files

Concern	Path
OAuth start/callback	`src/app/api/vendor/[vendorId]/square/oauth/start/route.ts`, `src/app/api/integrations/square/oauth/callback/route.ts`
Connection lifecycle	`src/lib/integrations/square/square-connection.service.ts`
Token crypto	`src/lib/integrations/integration-token-crypto.ts`, `integration-token-storage.service.ts`
Catalog import	`src/lib/integrations/square/square-menu-import.service.ts`, `square-catalog-normalizer.ts`
Mappings	`src/lib/integrations/provider-mapping.service.ts`

Readiness	`src/lib/integrations/square/square-order-routing-readiness.ts`
Submit	`src/services/square-order.service.ts`, `square-order-mapper.ts`, `square-api.client.ts`
Routing dispatch	`src/services/routing.service.ts`, `post-payment.service.ts`
Status sync	`src/services/square-status-sync.service.ts`, `square-status-mapper.ts`, `src/app/api/webhooks/square/route.ts`
Kitchen lock / copy	`src/lib/order-routing/kitchen-action-policy.ts`, `src/lib/integrations/provider-display.ts`
Admin diagnostics	`admin-square-order-injection-diagnostics.server.ts`, `square-mapping-diagnostics.server.ts`, vendor/order square-routing-debug routes
Env	`src/lib/env.ts` (`ENABLE_SQUARE_INTEGRATION`, `SQUARE_*`, `INTEGRATION_TOKEN_ENCRYPTION_KEY`)

What is fully built

Confidence: **High** = code + tests; **Medium** = code clear, limited E2E; **Low** = untested / conditional.

Feature	Evidence	Tests	Confider
OAuth connect with signed state + replay nonce	start/callback routes, `square-oauth-state.ts`, `square-oauth-nonce.service.ts`	`square-oauth-state.test.ts`, oauth route tests	High
Encrypted token storage + refresh-before-expiry	credential model + `ensureSquareAccessToken`	connection service tests	High
Merchant + location discovery/selection	`completeSquareOAuthForVendor`, `selectSquareLocationForVendor`	connection tests	High
Square → OO catalog import + publish	`SQUARE_CATALOG_PULL`, `square_catalog_v1`, publish soft-disable	menu import/normalizer/publish-guard tests	High
Location-scoped ProviderEntityMapping upsert/deactivate-not-seen	`syncSquareCatalogMappings`	menu import tests	High
Post-payment Square CreateOrder +	`submitVendorOrderToSquare`	`square-order.service.test.ts`	High

EXTERNAL CreatePayment			
Deterministic idempotency keys per VO	`oo:sq:order:{id}`, `oo:sq:pay:{id}`	order service tests	High
Payment-only retry when order exists	payment-only path in submit	order service tests	High
Multi-vendor independent routing by `vendorId`	post-payment loop + connection by VO vendor	architectural (partial dedicated isolation tests)	Medium-High
Webhook signature verify + event_id dedupe	webhook route + `ProviderWebhookEvent` unique	webhook verify + route tests	High
Status map + monotonic merge	`square-status-mapper.ts`, status sync service	mapper + sync tests	High
Kitchen action lock for Square-managed VOs	`getKitchenActionPolicy`	kitchen-action-policy tests	High
Ready-banner removal; "Managed in Square" compact badge	`vendorKitchenModeNotice` returns null when ready; `getKitchenManagedOrderBadge`	provider-display / kitchen UI tests	High
Admin injection + mapping diagnostics JSON	`/admin/vendors/[id]/square-routing-debug`, order debug	diagnostics tests	High
Global kill switch `SQUARE_ROUTING_LIVE`	readiness + submit	readiness tests	High

---

What is partially built

Item	Exists	Missing	Consequence	Sever
Injection readiness	Connection, scopes, location, Square menu kind, **any** item mapping	Full coverage of available MenuItems / required modifiers; cross-location drift blocker	Pay-then-fail	**Critic**
Location change	Select/store location; validate against location list	Invalidate/deactivate other-location mappings; vendor warning; require re-import	Poke Sea failure class	**Critic**

Mapping failure diagnostics	Per-line `squareLastError`; admin mappingFailureDiagnostics on readiness phrase	Always attach rich mappingFailureDiagnostics on mapper failures; normalized error codes	Ops harder to triage	High
Connection uniqueness	Soft-disconnect keeps inactive rows; upsert updates active	Unique partial index on active `(vendorId, provider)`; consistent `orderBy` on all finds	Rare wrong token/location	High
Status sync	Webhooks + admin manual sync	No background poller if webhooks misconfigured; only `order.updated`	Silent staleness	Mediu
Kitchen UX	Orders monitor framing, locked actions, compact badge	Optional hide of kitchen route for Square; leftover "Kitchen" copy in some nav/setup CTAs	Mild vendor confusion	Mediu
Money lifecycle	EXTERNAL payment + total reconciliation warn/block	Refund/cancel reverse on Square; tip visibility in Square	Accounting mismatch in POS	High
Public orderability vs Square ready	Setup checklist shows routing ready	`getVendorOrderabilityState` **intentionally** does not block when `squareOrderRoutingReady: false`	Vendor sellable while injection dead	**Critic**

What has not been built

- OO → Square catalog **push** / menu_publish to Square (capability enum exists; no exporter).
- Multi-location concurrent operation per vendor (design is **one active location**).
- Automatic Square refund / EXTERNAL payment reverse on OO refund.
- Square CancelOrder / void flows from Open Order.
- Checkout-time Square preflight blocking payment when mappings incomplete.
- Recurring health job that fails open/closed based on mapping drift.

- Deterministic DB unique constraint guaranteeing one active Square connection.
- Customer-facing messaging specific to “queued to Square / failed to Square” beyond generic routing failure UX (partial via issues/admin).
- Tip / service-fee line representation in Square orders.

End-to-end flow review

Connection → menu

1. Vendor with `canManageVendor` starts OAuth (`oauth/start`).
2. Callback binds **state.vendorId** (not attacker-controlled query vendor) after session match + replay consume.
3. Tokens encrypted; connection stores `externalMerchantId` , optional `externalLocationId` .
4. Multi-location → pending until `selectSquareLocationForVendor` (live list check when token present).
5. Catalog import pulls Square Catalog API → canonical menu `sourcePayloadKind: square_catalog_v1` → upsert PEM at **current location**.
6. Publish applies to live `MenuItem` rows (Square ids in `deliverectProductId`); unseen products → `isAvailable: false` .

Paid order → Square

1. Stripe payment succeeds → `post-payment.service` submits each pending VO.
2. Square VOs call `submitVendorOrderToSquare(vendorOrderId)` .
3. Already sent short-circuit if `routingStatus=sent` + `squareOrderId` .
4. `assertSquareOrderRoutingReady(vendorId)` (live + prereqs).
5. Map lines with PEM at connection location; fail → `recordSquareRoutingFailure` + issue.
6. CreateOrder → persist partial `squareOrderId` → EXTERNAL CreatePayment → success `sent` + map `vendor_order` .

Status return

1. Square `order.updated` webhook verified → deduped → fetch live Square order → map → monotonic apply to that VO.
2. Parent multi-vendor order completion is derived from sibling VOs (Square completion does not blindly complete other vendors).

Multi-vendor isolation review

Conclusion: Intended isolation is sound and appears correct for the Poke Sea / Iron Skewer incident; residual risk is schema races and global VO lookup by Square order id — not a "wrong vendor connection" bug in the happy path.

Check	Result
VO → own `vendorId`	Yes
Connection via `getActiveSquareConnectionForVendor(vendorOrder.vendorId)`	Yes
Mappings include `vendorId` + `externalLocationId`	Yes
Parallel VOs in one cart	Independent submits in post-payment loop
Shared mutable singleton for merchant/location	Not found
Global active Square connection	**No**
Same merchant, two OO vendors	Allowed; distinguished by OO vendorId + separate connections
Failures on one VO don't resubmit others	Loop continues; already-sent skip

Proven by incident data (local diagnosis 2026-07-15): Iron Skewer (`MLV1ND28KEF9G` / `L7186BQTYQC6X`) succeeded; Poke Sea (`MLXJSJ4WYQJPD` / `LN7RT05NHEW13`) failed on its own mappings — not cross-wired credentials.

Gaps: no unique active connection; `findVendorOrderIdBySquareOrderId` is global `findFirst` .

Catalog and location-drift review

Poke Sea (order `cmrh09p0j000lieq6vqgn0raw`)

Verified in local diagnostics / prior report (not re-asserted against production):

Fact	Detail
Parent order	`cmrh09p0j000lieq6vqgn0raw` (2026-07-11)

Iron Skewer VO	`confirmed` , Square order created
Poke Sea VO	`failed`
Error stored	Per-item mapper: `No active Square mapping for menu item "…"` (×9) — **not** readiness "No active Square item mappings for the selected location."
Selected location	`LN7RT05NHEW13`
Ordered product PEM	All nine mapped only at **`LNQCZRWXMCFE2`**
Later publish	New catalog IDs at new location; old MenuItems `isAvailable: false`
Current health (at diagnosis time)	New orders OK; **stale mappings still active** at old location (`mappingsExistForAnotherLocation: true`)

Can it still happen?

Yes. Readyness still:

1. Does not require every available item mapped.
2. Does not fail if mappings exist only on another location.
3. Does not clear other-location mappings on location change.
4. Does not block customers when `squareOrderRoutingReady` is false.

Strongest prevention

1. On location change / before routing live: deactivate PEM where `externalLocationId ≠ selected` .
2. Readiness: % coverage of available MenuItems (+ required modifiers) at selected location ≥ 100%, and `!mappingsExistForAnotherLocation` (or treat as hard warn blocking go-live).
3. Gate `getVendorOrderabilityState` on Square injection **prerequisites** (connection + location + menu + coverage).
4. Cart/checkout warn or block adding unmapped Square items.

Order and payment review

Topic	Behavior
Merchant of record	**Stripe** (customer charge)
Square role	Fulfillment / POS visibility via **EXTERNAL** tender

Payment request	`source_id: "EXTERNAL"`, `external_details.source: "Open Order"`, amount = Square order `total_money`
Tips / OO service fee	Not sent as Square line items
Tax	Square catalog tax rules; compared to OO `subtotal+tax` via reconciliation helpers
Refs	`reference_id = vendorOrder.id`; source name `"Open Order"`
Idempotency	Deterministic per VO; Stripe retry-safe for CreateOrder/Payment
Partial multi-vendor	Supported: one VO `sent`, another `failed`
Refunds → Square	**Not implemented**

Accounting risk: Square sales/EXTERNAL totals may not match Stripe net; tips may be invisible in Square; refunds leave Square paid unless staff adjusts manually.

Status-sync review

Transport: Webhooks (`order.updated` only) + admin manual pull. No scheduled poller.

Signature: HMAC over `notificationUrl + body` (`SQUARE_WEBHOOK_SIGNATURE_KEY`). Missing key → 503.

Status mapping table

Square fulfillment state	OO `fulfillmentStatus`	OO `routingStatus` (typical)
`PROPOSED`	`accepted`	`confirmed`
`RESERVED`	`preparing`	`confirmed`
`PREPARED`	`ready`	`confirmed`
`COMPLETED`	`completed`	`confirmed`
`CANCELED` / `CANCELLED` / `FAILED`	`cancelled`	`confirmed`

Square order state (fallback)	OO fulfillment
`COMPLETED`	`completed`
`CANCELED / CANCELLED`	`cancelled`

`OPEN`	(no map from order state alone)

Monotonic merge: terminal completed/cancelled sticky; backward non-terminal ignored (`ignored_backward`). Skipped Square states map to nearest OO step without requiring intermediate button presses in OO.

OO → Square cancel: not implemented as API call.
SMS duplicate risk: governed by OO notification plumbing + status merge (not Square-specific double-apply if webhook dedupe works).

Security review

Finding	Severity	Notes
OAuth state HMAC + TTL + DB nonce replay	—	Good
Callback requires session user = state user + `canManageVendor`	—	Good tenancy
Tokens AES-GCM at rest	—	Good; key must be set (`INTEGRATION_TOKEN_ENCRYPTION_KEY` / AUTH_SECRET fallback)
Webhook signature required	—	Good when configured
Mixing sandbox/prod credentials	**High**	Mitigated by mismatch warnings; still operationally fragile
No unique active connection	**High**	Race → ambiguous token/location
Global `findVendorOrderIdBySquareOrderId`	**Medium**	Prefer squareOrderId + vendor/merchant
Secrets in admin JSON	—	Diagnostics designed to omit tokens; keep guarding
Debug oauth minimal scopes route	**Low–Medium**	Ensure debug not exposed in production misuse
AUTH_SECRET used for state signing	**Low**	Standard; rotate carefully

---

Data-integrity review

Concern	Detail
No `@@unique([vendorId, provider, isActive])` for active-only	Multiple actives possible
PEM unique includes nullable `externalLocationId`	PG allows multiple NULL location rows
`squareOrderId` indexed, not unique	Duplicate rows theoretically possible
Disconnect does not wipe mappings	Stale mappings accumulate across locations
MenuItem no unique on `(vendorId, deliverectProductId)`	Comments acknowledge possible duplicate rows
Credential hard-delete on disconnect	OK; historical tokens gone
Soft disconnect keeps connection history	Good for audit; requires careful "active" queries
Webhook event unique `(provider, externalEventId)`	Good

UX review

Vendor

Area	Current
Connect / reconnect / disconnect	`VendorSquareConnectionCard`
Location picker	Only when pending selection; weak post-select change UX
Catalog import	Square integrations + menu imports panels
Ready banners	Long "Square routing is ready… manage kitchen in Square" **removed** when operational
Kitchen	Titled **Orders monitor**; helper "Manage order status in Square."; actions locked once Square-managed
Failure visibility	Errors on VO; richer details mainly admin

Recommended final: require re-import after location change with blocking banner; keep Orders monitor read-only; surface "mappings at another location" to vendors, not only admins.

Admin

Strong: injection diagnostics, mapping-by-location, connection ids, readiness blockers, order Square debug, manual status sync, retry routing.
Gaps: always-on structured mapping failure payload; one-click "deactivate non-selected location mappings."

Customer

Sees normal OO status; post-pay Square failure becomes routing/issue path — risk of "paid but kitchen never got it" if ops miss alerts.

Test-coverage matrix

Executed this audit: 23 files, **127 passed** under `src/lib/integrations/square`, Square services, webhook route, kitchen-action-policy.

Area	Covered	Test files (examples)	Missing cases	Risk
OAuth state/nonce	Yes	`square-oauth-state`, oauth routes	Concurrent dual-callback race	Med
Location select	Partial	connection service	Location change → mapping invalidation E2E	**Crit**
Menu import/publish	Yes	menu-import, normalizer, publish-guard	Partial publish crash recovery	High
Mapping location scope	Partial	import + diagnostics	Ordered items mapped only on wrong location E2E	**Crit**
Order submit + EXTERNAL pay	Yes	`square-order.service.test`	Real Square sandbox timeout after CreateOrder	High
Idempotency / payment-only retry	Yes	order service tests	Concurrent double submit race harness	Med
Multi-vendor isolation	Partial	architecture + readiness; incident evidence	Automated parallel two-merchant mock	High
Webhooks + verify + dedupe	Yes	webhook route/verify	Out-of-order burst stress	Med
Status mapping / monotonic	Yes	status-mapper, status-sync	Skip PROPOSED→PREPARED	Low
Readiness count>0 gap	Indirect	readiness tests assert count gate	Coverage-% gate **not tested because absent**	**Crit**

Orderability vs Square ready	Documented	`vendor-readiness-states.test` **allows** orderable when routing not ready	Inverse gate desired	**Crit**
Refunds to Square	No	—	Entire area	High
Kitchen lock / monitor copy	Yes	kitchen-action-policy, provider-display	—	Low
Admin diagnostics	Yes	admin diagnostics tests	—	Low

Launch blockers

B1 — Block paid orders when Square injection prerequisites fail

- **Severity:** Critical
- **Scenario:** Vendor visible/orderable with bad/missing mappings; customer pays; VO `failed`.
- **Files:** `vendor-readiness-states` / orderability evaluation; checkout guards; `square-order-routing-readiness.ts`
- **Fix:** Require Square **prerequisitesReady** (or stronger coverage) for `orderable`.
- **Test:** Orderability false when mapping coverage incomplete; checkout rejects.

B2 — Full mapping coverage + selected-location only

- **Severity:** Critical
- **Scenario:** Poke Sea: mappings on old location; readiness passes if any mapping at new location; ordered SKUs miss.
- **Files:** `square-order-routing-readiness.ts`, `square-mapping-diagnostics.server.ts`, menu publish
- **Fix:** Ready only if every available item (+ required modifiers) has active PEM at selected location; fail if `mappingsExistForAnotherLocation` without cleanup.
- **Test:** Fixture with dual-location PEM; ordered old ids → not ready / cart block.

B3 — Location change / republish must reconcile mappings

- **Severity:** Critical
- **Scenario:** Location or catalog IDs change; stale PEM remain active.
- **Files:** `selectSquareLocationForVendor`, `disconnect`/reconnect, `syncSquareCatalogMappings`

- **Fix:** On location change: deactivate non-selected location PEM; force re-import+publish before go-live.
- **Test:** Select new location → old PEM inactive; readiness false until re-import.

B4 — Unique active Square connection per vendor

- **Severity:** High (launch if multi-reconnect expected)
- **Scenario:** Two active rows → nondeterministic `updatedAt` winner.
- **Files:** schema, `upsertSquareConnection`, `getActiveSquareConnectionForVendor`
- **Fix:** Partial unique index; transactionally deactivate siblings on upsert.
- **Test:** Concurrent upsert leaves one active.

B5 — Persist structured mapping failure diagnostics on mapper failures

- **Severity:** High (ops)
- **Scenario:** Per-item string errors without connection/location/missing ids always attached.
- **Files:** `square-order.service.ts`, audit types, admin panel
- **Fix:** Always write `mappingFailureDiagnostics` (and normalized code) on VALIDATION_FAILED / readiness mapping blockers.
- **Test:** Failed submit audit contains vendorId, locationId, missing item ids.

Post-launch improvements

- Square cancel/refund money sync or documented ops SOP.
- Optional status poller for webhook silence.
- Vendor-visible drift warnings (not admin-only).
- Tip presentation in Square order notes.
- Hide Kitchen nav entirely for Square mode (Orders ledger only).
- Scope webhook VO lookup by merchant/connection.
- Chaos tests: timeout after CreateOrder, duplicate webhooks, sandbox/prod mismatch.
- MenuItem uniqueness on Square product id per vendor.

Recommended implementation plan

Sprint 1: Prevent paid-order routing failures

Task	Goal	Likely files	Schema	Tests	Ac
S1.1 Coverage readiness	Ready iff 100% available items (+ mods) mapped at selected location	`square-order-routing-readiness.ts`, diagnostics	none	coverage fixtures	Re wit

					un ite
S1.2 Cross-location hard fail	Ready false if item PEM only on other locations / flag block	readiness + diagnostics	none	dual-location fixture	Po pro rea
S1.3 Orderability gate	Public orderable requires Square prereqs	vendor readiness / checkout	none	invert current readiness-states test	Ca ch wh rea
S1.4 Location-change cleanup	Deactivate non-selected PEM; require re-import	`square-connection.service.ts` , import	optional job	location-change test	Ol loc PE ina
S1.5 Unique active connection	One active Square connection	schema migration, upsert	partial unique	race test	Sin act
S1.6 Structured errors	Always store mappingFailureDiagnostics	`square-order.service.ts`	none	audit shape test	Ad sh mi

Sprint 2: Reliable Square lifecycle

Task	Goal	Likely files	Schema	Tests	Acceptance
S2.1 Token health alerts	Visible vendor+admin when refresh fails	connection health, UI	none	refresh-fail fixture	Banner + not orderable
S2.2 Republish atomicity	Publish cannot leave half-mapped live menu	publish services	optional revision	failed publish rolls back	No orphan available items
S2.3 Webhook resilience	Alert on silence; optional poller for stuck sent VOs	status-sync, cron	optional	webhook down sim	Stuck VO flagged
S2.4 Safer VO resolution	Resolve Square order id + vendor/merchant	status-sync	none	two VO same external id if possible	Correct VO only
S2.5 Timeout after CreateOrder	Documented + tested payment-only recovery	order service	none	timeout mock	No duplicate Square orders

Sprint 3: Operational tooling and UX

Task	Goal	Likely files	Schema	Tests	Acceptance
S3.1 Vendor drift UI	Show cross-location / unmapped counts	Square cards	none	UI tests	Vendor must re-import to clear
S3.2 Admin one-click cleanup	Deactivate other-location mappings	admin actions	none	action test	Flag clears
S3.3 Kitchen product finish	Hide kitchen nav for Square; ledger-first	VendorAreaNav, ledger	none	nav tests	No kitchen actions
S3.4 Refund/cancel SOP or sync	Decision + implement or document	refunds services	maybe	—	Ops-approved
S3.5 Observability pack	Dashboard fields: merchant, location, attempts, idempotency, Square id	admin panels, logs	none	—	Support can answer checklist questions

---

Final readiness checklist

Use before enabling `SQUARE_ROUTING_LIVE=true` for real vendors:

- ☐ Production Square app credentials; `SQUARE_ENVIRONMENT=production`; no sandbox mix
- ☐ `ENABLE_SQUARE_INTEGRATION=true`
- ☐ `INTEGRATION_TOKEN_ENCRYPTION_KEY` set (≥32)
- ☐ OAuth redirect URL exact match
- ☐ Webhook subscription `order.updated` + `SQUARE_WEBHOOK_SIGNATURE_KEY` + matching notification URL
- ☐ Required OAuth scopes approved (ORDERS_WRITE, PAYMENTS_WRITE, …) — vendors reconnected after scope expansion
- ☐ Each pilot vendor: one active connection, correct merchant + location
- ☐ Catalog imported **and published** as `square_catalog_v1`
- ☐ **Zero** active item mappings on non-selected locations
- ☐ **100%** available MenuItems (+ required modifiers) mapped at selected location
- ☐ Orderability blocked when Square prereqs fail (code change from B1)
- ☐ Multi-vendor sandbox order: two Square merchants succeed independently

- [ ] Forced mapping failure path: customer cannot pay / or clear rescue path tested
- [ ] Status: Square PREPARED → OO ready; COMPLETED → completed; cancel → cancelled
- [ ] Admin can see Square order id, attempts, last error, location, missing mapping ids
- [ ] Kitchen/Orders monitor is read-only for Square-managed VOs
- [ ] Ops runbook for refunds (Stripe vs Square EXTERNAL) signed off
- [ ] `SQUARE_ROUTING_LIVE=true` only after above

Appendix A — Env vars (Square)

Variable	Role
`ENABLE_SQUARE_INTEGRATION`	Gate connect UI in production
`SQUARE_APPLICATION_ID / SECRET`	OAuth app
`SQUARE_ENVIRONMENT` / `SQUARE_MODE`	sandbox | production
`SQUARE_OAUTH_REDIRECT_URL`	Callback
`SQUARE_ROUTING_LIVE`	Live CreateOrder/Payment kill switch
`SQUARE_WEBHOOK_SIGNATURE_KEY`	Webhook HMAC
`SQUARE_WEBHOOK_NOTIFICATION_URL`	Signature URL base
`SQUARE_TOTAL_MISMATCH_WARN_CENTS / BLOCK_CENTS`	Total reconciliation
`INTEGRATION_TOKEN_ENCRYPTION_KEY`	Token AES key
`AUTH_SECRET`	OAuth state signing (+ optional crypto fallback)

Appendix B — Facts vs recommendations

Statement	Type
Poke Sea failed on per-item mappings at wrong location	**Fact** (local diagnosis)
Readiness uses `activeItemMappingCount > 0`	**Fact** (`square-order-routing-readiness.ts`)
Orderability ignores `squareOrderRoutingReady: false`	**Fact** (`vendor-readiness-states.test.ts`)

Ready Square kitchen banner removed	**Fact** (`vendorKitchenModeNotice`)
Square is not MoR; Stripe is	**Fact** (submit service contract)
Should block checkout until coverage 100%	**Recommendation** (blocker B1/B2)
Should unique-index active connections	**Recommendation** (blocker B4)

End of audit. PDF sibling generated via `npm run report:pdf` .